

Lista de Exercícios: Tratamento de Erros

Programação de Computadores em Python — Exceções e Sintaxe

TÓPICOS COBERTOS

- Diferença entre erros de sintaxe (SyntaxError) e exceções em tempo de execução (Runtime Errors).
- Estrutura básica do bloco try e except.
- Captura de exceções específicas (ex: ValueError, ZeroDivisionError, TypeError).
- Captura de múltiplas exceções e uso da alias (as e).

EXERCÍCIO 1

Conceitos e Identificação de Erros

Classifique os trechos de código abaixo indicando se eles causam um **Erro de Sintaxe (SyntaxError)** ou uma **Exceção em Tempo de Execução (RuntimeError)**:

a)

```
if x > 10
    print("Maior que 10")
```

b)

```
numero = int("cem")
```

c)

```
resultado = 10 / 0
```

d)

```
def minha_funcao()
    return "Olá"
```

EXERCÍCIO 2

Estrutura Básica do try e except

Escreva um programa em Python que peça ao usuário para digitar um número inteiro. Use o bloco `try` e `except` para capturar qualquer erro na conversão e exibir a mensagem:

"Entrada inválida! Por favor, digite apenas números."

caso a conversão falhe.

EXERCÍCIO 3

Captura de Exceções Específicas

Crie uma calculadora simplificada de divisão que solicite dois valores ao usuário e exiba o resultado da divisão do primeiro pelo segundo. Trate especificamente os seguintes erros com mensagens personalizadas:

- **ValueError**: Caso o usuário digite algo que não possa ser convertido para número (float ou int).
- **ZeroDivisionError**: Caso o usuário tente dividir por zero.

EXERCÍCIO 4

Múltiplas Exceções e Uso de Alias (as e)

Escreva uma função chamada `somar_elementos(lista, indice_1, indice_2)` que recebe uma lista de dados e dois índices para somar os valores presentes nessas posições.

1. Tente realizar a soma dos elementos apontados por `indice_1` e `indice_2`.
2. Trate de forma independente as exceções:
 - **IndexError**: Se algum dos índices estiver fora do limite da lista.
 - **TypeError**: Se os elementos nas posições indicadas não puderem ser somados (ex: somar uma string com um número).
3. Utilize o recurso `as` e para capturar a mensagem nativa do erro e exibi-la formatada no console (exemplo: `print(f"Erro capturado pelo Python: {e}")`).